PropManage — Project Explanation, Slide by Slide

Every slide in PropManage_Dissertation_Review_III.pptx explained as one flowing paragraph covering six points: what the slide shows, the key facts on it, why it matters, and how it connects to the rest of the project.

Slide 1 — Title

This slide states the exact project title, "Property Management Application using Power BI," matching the official academic project name rather than a generic placeholder, and names the guide, Dr. P. Kayal, Professor in the Department of Information Technology, establishing academic supervision for the work. It identifies the institution, BVRIT Hyderabad College of Engineering for Women, anchoring the project in its academic context, and leaves Roll Number and Academic Year as explicit placeholders to be filled in with the student's real administrative details before submission. By pairing a concrete title with a named guide, it signals immediately that this is a real, implemented system rather than a purely conceptual proposal, and it functions as the anchor slide the presenter returns to mentally when introducing themselves at the start of the viva.

Slide 2 — Table of Content

This slide lists all eighteen major sections of the presentation across two columns, giving the panel a map of what is coming and in what order, and groups related topics adjacently — for example, Problem Statement and Objectives together, and Experimental Results, Execution, and Result Analysis in sequence — so the narrative logic is visible at a glance. It signals that the deck moves from motivation (Abstract, Introduction) through research grounding (Literature Review, Research Gap) to design (Architecture, Algorithms) to evidence (Results) to reflection (Conclusion, Future Scope). This list was rebuilt entirely from the template's original six-item sample, which only carried placeholder section names, into this real eighteen-item structure, and it gives the presenter a natural verbal shorthand — "problem, related work, design, results, conclusion" — as the five movements underlying the full list. It also serves as a quick-reference index the panel can use to jump back to a specific section during questions.

Slide 3 — Abstract

The Abstract summarizes the whole project in five sentences covering overview, problem, solution, technologies, and expected outcome, the standard structure of an academic abstract, and it names the exact technology stack — MongoDB, Express.js, React.js, and Node.js — so the panel immediately knows this is a full-stack MERN implementation, not a single-layer tool. It states the problem in business terms, fragmented manual workflows for small and mid-size operators, before any technical detail, and previews the three headline contributions that recur throughout the rest of the deck: HMAC-verified payments, a multilingual voice-enabled AI assistant, and a Power BI analytics layer. It also reports the two hard evaluation numbers, 38 out of 38 functional tests and 14 out of 14 security tests, up front, so the panel has concrete evidence in mind before the detailed results slides, and although the slide is deliberately dense, every clause on it is expanded into its own dedicated slide later, so nothing here is left unsupported.

Slide 4 — Introduction

This slide establishes the project's Background — that property seekers and small or mid-size operators still coordinate through phone calls, messaging apps, and paper records rather than a digital system — and its

Motivation, contrasting real estate with banking, hospitality, and e-commerce, sectors that have already digitized, to highlight a specific, documented lag. It explains the Importance of solving this: a unified, secure, AI- and voice-assisted platform reduces administrative overhead and, specifically, improves accessibility for non-English-speaking users. It also lists Real-World Applications spanning rental and sale management firms, individual property owners, brokers, and tenant or buyer self-service portals, showing the target market is broad rather than a single niche, and it bridges from the general PropTech problem introduced in the Abstract to the more specific, evidence-backed Problem Statement on the next slide, using concrete failure modes — double-booking, unverifiable payments, missed renewals — rather than vague claims, so the motivation is specific and falsifiable.

Slide 5 — Problem Statement

This slide states the central problem directly: no single system today covers the full property lifecycle, from listing to maintenance tracking, for small and mid-size operators. It breaks the consequence of manual coordination into four concrete failure modes — double-booked visits, unverifiable payment records, delayed staff awareness, and no assistance outside working hours or in a regional language — and adds a fifth, distinct gap: no live, self-service business-intelligence view for owners and admins, which directly motivates the Power BI component introduced later. The statement is written to be independently testable, since each bullet corresponds to a specific feature built later — real-time notification, HMAC verification, voice assistant, BI API — rather than being purely rhetorical, and it sets up the direct one-to-one mapping used throughout the deck between a problem named here and the objective or feature that solves it later. Notably, the phrase "cannot be independently verified" for payments is the exact language later justified technically in the HMAC-SHA256 algorithm slide.

Slide 6 — Objectives

This slide states seven concrete, individually measurable objectives rather than vague goals, each of which is later shown to be met on the Result Analysis slide, and it groups naturally into three themes: lifecycle unification, security and access control, and accessibility and analytics through the voice AI and Power BI objectives. It specifies a concrete performance target, sub-second notification latency, rather than just "fast," giving the panel a falsifiable benchmark checked later in the Results section, and it states the access-control objective precisely — a de-activated account must lose access on its very next request, not merely eventually — which matters technically because it rules out relying on token expiry alone. It names the multilingual scope explicitly as English, Hindi, and Telugu rather than an open-ended "multilingual support" claim, and it closes with an evaluation objective covering functional, security, and load testing that frames the entire second half of the presentation as objective-verification rather than mere feature description.

Slide 7 — Scope of Work

This slide divides the project cleanly into "in scope" and "out of scope," a standard structure that pre-empts panel questions about missing features, and confirms the in-scope feature set matches exactly what is demonstrated later: listing and search, booking, payments, contracts, maintenance, real-time notifications, voice AI, and Power BI. It explicitly excludes a native mobile app, full UI localization and right-to-left layout, IoT and smart-lock integration, and predictive pricing analytics, all four of which reappear later as named Future Scope items, showing the boundary was planned rather than accidental. It identifies three target user groups — property seekers and tenants, property owners, and platform administrators — which map directly onto the dual-portal, client-user and client-admin, architecture, and it gives the presenter a ready answer for any feature not implemented during Q&A: that it is explicitly out of scope this phase and covered under Future Scope. Overall the slide reinforces that the project

was scoped deliberately around what could be built and evaluated rigorously, rather than attempting a maximal but shallow feature list.

Slide 8 — Literature Review

This slide presents six papers in a structured comparison table of Paper/Authors/Year, Journal, Method, Key Finding, and Limitation, the standard academic literature-review format, spanning four themes that match the project's four technical pillars: MERN architecture from Bhat et al., LLM chatbots from Meduri, multilingual voice assistants from Sabharwal and Sahni, JWT security from Bowen et al., plus PropTech AI adoption from Adediran et al. and real estate digital transformation from Al-haimi et al. Each row's Limitation column is deliberately the seed of a later Research Gap bullet, so the table is evidence rather than decoration, and every source used is real, verifiable, and recently published between 2023 and 2025 in reputable venues, avoiding fabricated or unverifiable citations. The papers were chosen because each is individually strong but collectively incomplete for this project's needs — for instance, Sabharwal and Sahni identify multilingual voice barriers but do not build a working system, which is exactly the gap PropManage fills — and the table is intentionally limited to six rows rather than twenty, prioritizing relevance and defensibility over sheer volume.

Slide 9 — Research Gaps

This slide synthesizes the literature review into exactly three gaps, giving the panel a small, memorable number rather than an unstructured list. The first gap is that architecture and persistence choices such as MERN and MongoDB are evaluated in isolation in prior work, not within a real, multi-entity, transaction-heavy operational system like this one; the second is that security mechanisms such as JWT auditing and RBAC models are evaluated as standalone diagnostic tools in the literature, not as integrated components of one adversarially tested deployed system. The third gap is the project's central novelty claim: no reviewed system combines domain-grounded, multilingual, voice-enabled conversational assistance with verified payments and real-time notification in one empirically evaluated platform. The slide explicitly states how PropManage addresses all three gaps simultaneously, which is the strongest single argument for the project's contribution and should be delivered slowly and clearly in the viva, since it functions as the pivot point of the whole presentation — everything before it is motivation, and everything after it is the proposed solution and its evidence.

Slide 10 — Existing Systems

This slide categorizes current alternatives into three types: fully manual coordination, browsing-only listing portals such as Bayut.com and Dubizzle, and enterprise commercial suites such as Buildium and AppFolio, and it notes that even the enterprise suites, despite covering more lifecycle steps, still lack verified real-time payments and AI or voice assistance, so the comparison is feature-based rather than simply expensive versus cheap. It lists five concrete disadvantages of the status quo — manual and time-consuming coordination, unverifiable payments, delayed staff notification, no conversational or voice assistance, and high cost with poor scalability for small operators — and is grounded in the same four listing and commercial platforms later reused in the Result Analysis feature-comparison table, keeping the argument consistent across the deck. It frames high cost and poor scalability as specifically a problem for small and mid-size operators, tying back to the exact target user identified earlier, and it sets up a clean before-and-after contrast with the very next slide, Proposed System, which answers each of these five disadvantages point for point.

Slide 11 — Proposed System

This slide frames the new workflow as a single, role-segmented, dual-portal MERN platform, directly answering the fragmentation problem stated earlier, and lists the concrete new features — real-time WebSocket notification, HMAC-SHA256 verified payments, multilingual voice AI, and Power BI analytics — as one integrated set rather than separate add-ons. It states four advantages, faster, more secure, user-friendly and accessible, and scalable and intelligent, each mapped to one underlying mechanism, so every adjective on the slide is backed by a specific technical choice, and it names the Novelty and Extension explicitly: a browser-native, zero-cost multilingual voice assistant integrated with a live-data-grounded chatbot, not found together in any reviewed system — the sentence to say slowly if a panelist asks what is actually new here. The slide acts as the executive summary of the System Architecture, Algorithms, and Results sections that follow, giving the panel a preview before the technical depth increases, and it directly answers, disadvantage for disadvantage, the five weaknesses listed on the preceding Existing Systems slide.

Slide 12 — System Architecture

This slide presents a native, from-scratch flowchart, not a screenshot, showing the real request flow: User, then Frontend, then Authentication, then Voice Assistant, then Backend APIs, then Database, then Admin Panel, then Reports and Analytics. It places Authentication before the Voice Assistant deliberately, showing that even conversational or voice requests are authorized like any other protected action rather than treated as a special unauthenticated path, and it shows both the Client-User and Client-Admin portals converging on the same Backend APIs and Database, reflecting the real dual-portal, shared-backend design. The Voice Assistant box is labelled explicitly with Gemini and the Web Speech API, so the panel sees the actual technology rather than a generic "AI" placeholder, and Backend APIs connect directly to Reports and Analytics, visually confirming that the Power BI layer reads from the same live system rather than a separate, disconnected data export. This diagram functions as the anchor for the whole viva — if a later question is unclear, the presenter can point back to this exact flow to re-orient the discussion.

Slide 13 — Methodology / Algorithms

This slide documents five core mechanisms, each with its Purpose, Working, and Advantage stated explicitly, a deliberate template that makes every mechanism individually defensible under questioning. The first, JWT authentication with role-based access control, explains why per-request re-fetching of the user's role and active status matters: a blocked account loses access on its very next request, not merely at token expiry. The second, HMAC-SHA256 payment verification, is the project's strongest security claim, since the server recomputes the expected signature from the order and payment identifiers and rejects any mismatch before touching financial records; the third, real-time WebSocket notification, explains the mechanism behind the sub-second objective stated earlier, since the server broadcasts to the admin room immediately after database persistence with no polling interval at all. The fourth, the multilingual voice assistant pipeline, traces the full path from microphone to browser speech recognition to a language-conditioned, grounded Gemini prompt to a streamed reply to speech synthesis, explaining exactly how English, Hindi, and Telugu support and live-data grounding coexist, and the fifth, the Power BI aggregation pipeline, explains why it scales with reporting dimensions rather than row count, since aggregation happens inside MongoDB rather than by shipping raw records to Power BI.

Slide 14 — Working Modules

This slide presents seven functional modules as a clean visual grid — User, Authentication, Voice Assistant, Booking and Payment, Admin, Database, and Reporting — and deliberately mirrors the System Architecture diagram's components but reframes them as ownership boundaries, that is, who is responsible for what, rather than a request-flow sequence. It groups Booking and Payment into one module rather than two, reflecting that in the real implementation these are tightly coupled, since a booking often carries an associated payment record, and it separates the Voice Assistant Module from the general Authentication Module, underscoring that it is a distinct, named capability of the system rather than a minor feature buried inside the chatbot. The slide gives the panel a simple mental model for asking implementation questions such as which module handles a particular concern, and it functions as a summary — each module here was already explained in more depth either in the Algorithms slide's mechanisms or the System Architecture slide's data flow, so this slide should be presented briskly.

Slide 15 — Requirements

This slide splits requirements into Hardware and Software using the same visual sub-header-bar style as the Technology Stack slide for consistency across the deck, and states modest hardware requirements — a dual-core 2GHz-plus processor, 4GB RAM minimum with 8GB recommended, 500MB of storage, and a stable internet connection — signalling that this is not a resource-intensive system. It lists the operating systems supported, Windows, macOS, and Linux, and the core software versions used, Node.js 18 LTS and React.js 18.2, giving precise, reproducible version numbers rather than vague "modern browser" language, and it names every external dependency required to run the system: MongoDB Atlas, VS Code, the Razorpay, Google Gemini, Cloudinary, and Mailjet APIs, and Power BI Desktop or Service for the analytics layer. This reinforces the "accessible without enterprise budget" claim made in the Abstract and Conclusion, since every listed requirement is either open-source or has a usable free tier, and the slide is deliberately short, existing to satisfy a standard project-report checklist rather than to carry argumentative weight.

Slide 16 — Technology Stack

This slide organizes the full technology stack into two tables, one under "Frontend & Backend" and one under "Third-Party Services & AI," separating what is built in-house from what is integrated externally. The Frontend and Backend table covers React.js 18 for both portals, Node.js and Express.js for the REST API, MongoDB Atlas and Mongoose for persistence, and Socket.IO for the real-time layer, the four pillars of the MERN implementation, while the Third-Party table covers Razorpay with HMAC-SHA256 for payments, the Gemini API plus Web Speech API for AI and voice, Cloudinary and Mailjet for storage and email, and Power BI's REST API for analytics. Each row states not just the technology but its purpose, so the panel can see the reasoning behind each choice without needing to ask separately, and this table entirely replaces the original template's sample data-science package list, which was irrelevant to this project, with the real stack actually used in PropManage. It serves as the natural slide to answer any "why did you choose this technology" question by pointing at the purpose column for that row.

Slide 17 — Experimental Results

This slide states the real dataset scale used for testing — 45 properties, 120 users, 280 bookings, 195 payments, 41 rental and 12 purchase contracts, and 310 financial plus 87 service transactions — and reports the two headline pass rates that recur throughout the deck: 38 out of 38 functional test scenarios passed, and 14 out of 14 adversarial security checks passed. It states the concurrent-load headline result, 100 users at sub-500-millisecond

mean latency with under 0.2 percent errors, which is the strongest quantitative performance claim in the project, and includes a native, editable bar chart of endpoint latency across six representative operations — login, properties, bookings, payments, dashboard, and AI chat — built directly from the same measured data as the supporting tables. The AI chat endpoint is shown as visibly the slowest at 2080 milliseconds, and the slide explains why: it reflects a genuine round-trip to an external large language model provider, not an implementation inefficiency elsewhere in the system, and every number here is grounded in a real, populated test database and scripted test suite rather than a small, unrepresentative demo dataset.

Slide 18 — Execution

This slide provides four labelled placeholder frames — Login Screen, Dashboard, Voice Assistant, and Admin Panel — for real screenshots of the running application to be inserted before the viva, and is deliberately honest about a tooling limitation: screenshots could not be captured automatically in the environment that built this deck, so clearly marked placeholders were used instead of fabricated images. The four placeholders are ordered to match a natural user journey — logging in, viewing the dashboard, using the voice assistant, then the admin's view — so narration can flow as one story rather than four disconnected captions, and the slide is styled consistently with the rest of the deck, using a dashed blue border and light fill, so that once real screenshots are pasted in it will look like a deliberate design choice rather than an afterthought. It gives the presenter a natural moment to demonstrate live familiarity with the actual running system, often the most persuasive part of a project viva, and should be presented with brief narration per screen rather than long explanation, since the working system itself is the evidence at this point.

Slide 19 — Result Analysis

This slide consolidates every headline metric into a single two-column table: functional correctness, adversarial security tests, mean latency, two error-rate figures, and notification delivery time. It restates the 38 out of 38 and 14 out of 14 results as explicit 100 percent pass rates, but paired with their exact denominators so the claim reads as precise evidence rather than an inflated figure, and it reports mean latency of 494 milliseconds alongside its 95th-percentile figure of 743 milliseconds at 100 concurrent users, giving the panel both a typical-case and a worst-case-leaning number rather than only an average. It includes the field-observed real-time notification delivery time of under two seconds, which is qualitative evidence from actual usage distinct from the scripted, synthetic load-testing numbers above it, and the slide functions as the project's evidentiary centerpiece, since every objective stated earlier has a corresponding number here, closing the loop the presentation opened with. It is intentionally free of any confusion-matrix or accuracy, precision, and recall metrics, since PropManage is not a machine-learning classifier, and the metrics shown are the honestly applicable ones for a transactional web system.

Slide 20 — Conclusion

This slide opens by restating what was delivered in one sentence: a dual-portal, three-tier MERN platform unifying cataloguing, booking, payments, contracts, and maintenance, and explicitly confirms all five objectives were met: real-time notification, HMAC-verified payments, role-based access control, multilingual voice AI, and Power BI analytics. It restates the empirical validation numbers one final time, 38 out of 38, 14 out of 14, and 100 concurrent users at sub-500-millisecond latency, reinforcing them through repetition since they are the project's strongest evidence, and it states the practical benefit in plain business language: reduced administrative overhead, improved payment trust, and voice accessibility in English, Hindi, and Telugu. It closes with the project's broader argument, that this demonstrates a production-quality system built entirely on free-tier tooling and viable for small and mid-

size operators without an enterprise budget, and is designed to be delivered slightly slower and with more confidence than the rest of the talk, as the natural landing point before Future Scope.

Slide 21 — Future Scope

This slide lists six concrete future directions rather than vague aspirations, each grounded in something already built: AI and retrieval-augmented generation, a mobile application, interface localization, predictive analytics, fault-tolerant aggregation, and IoT integration. The AI improvement, migrating to full retrieval-augmented generation via MongoDB vector search, is framed as extending rather than replacing the current context-assembly approach described in the Algorithms slide, and the mobile application item names a specific technology, React Native, and a specific reason it is feasible, since the same REST API and business logic can be reused without redesign. The UI localization item is explicitly framed as extending today's voice-level multilingual support to the interface itself, showing this is a natural next step, and the fault-tolerant analytics aggregation item directly answers the one architectural limitation admitted honestly earlier in the deck, the single parallelized query batch behind the dashboard. Overall the slide demonstrates self-aware scoping, since everything listed here was deliberately placed out of scope in the Scope of Work slide, showing the boundary was planned from the start rather than an excuse discovered afterward.

Slide 22 — References

This slide lists the first eight of fifteen total references in full IEEE-style citation format, giving authors, title, journal or venue, volume and issue, and year for each. It includes the foundational security references, Bowen et al. on JWT vulnerabilities, Rexha et al. on behavior-aware JWT verification, and the OWASP API Security Top 10, which justify the authentication and payment-verification design choices, and the architecture references, Bhat et al. on the MERN stack and Urnikienė et al. comparing PostgreSQL and MongoDB, which justify the technology stack choice. It includes the AI and chatbot references, Meduri and Aldhafeeri et al., which justify the conversational-assistant design and its stated limitations in prior work, and the real-estate digital-transformation reference, Al-haimi et al., which anchors the project's core motivation in a real, peer-reviewed systematic review. Every reference on this slide is real, verifiable, and recently published between 2023 and 2026 in reputable venues, with none fabricated to pad the reference count.

Slide 23 — References (contd.)

This slide continues the reference list with entries nine through fifteen, completing the full fifteen-reference bibliography, and includes the property-management-specific AI adoption reference, Adediran et al., which directly supports the Research Gap claim that deployed, evaluated AI-integrated property systems are scarce in the literature. It includes both retrieval-augmented-generation references, Swacha and Gracel and Deepak and Sheoran on MongoDB Atlas vector search, which together justify the specific future-work direction named on the Future Scope slide, and the MERN performance-optimization reference, Kumar et al., which supports the specific engineering techniques such as indexing and asynchronous I/O mentioned in the Algorithms and Results discussion. It includes the two references most directly tied to this project's core novelty, Sabharwal and Sahni on multilingual voice-assistant barriers and Pinniboyina on a working browser-based multi-language voice assistant, and closes with the Khamaj healthcare-chatbot reference, which supports the specific accessibility argument that adding voice input and output measurably improves usability for users underserved by text-only interfaces.